

ResNet

<https://arxiv.org/abs/1512.03385>

Abstract

- 문제의식: 네트워크의 깊이가 깊어질수록 표현력은 증가하지만, 실제 학습과 최적화는 점점 어려워짐.
- 개선 방법: 레이어들이 원하는 함수 $H(x)$ 를 직접 학습하도록 하지 않고, 입력 x 를 기준으로 필요한 변화량인 residual function $F(x)$ 를 학습하도록 함.

$$F(x) = H(x) - x$$

따라서 원래 학습하고자 했던 함수는 다음과 같이 표현됨.

$$H(x) = F(x) + x$$

- 입력 x 는 shortcut connection을 통해 그대로 전달
- convolution layer들은 입력을 완전히 새로운 값으로 변환하는 대신, 입력에 추가해야 할 변화량 $F(x)$ 만 학습
- 이러한 residual learning 구조를 통해 매우 깊은 네트워크도 안정적으로 최적화 가능
- ImageNet에서 최대 152-layer ResNet을 학습했으며, classification뿐 아니라 object detection에서도 기존 모델보다 높은 성능을 기록함

1. Introduction

1. 더 깊은 CNN이 필요한 이유

- CNN의 초기 레이어는 edge, texture, color와 같은 저수준 특징을 추출한다.
- 중간 레이어는 object part나 반복적인 형태를 학습한다.
- 깊은 레이어는 전체 object나 class와 관련된 고수준 의미 정보를 추출한다.
- 따라서 네트워크를 깊게 만들면 더 풍부하고 계층적인 feature representation을 학습할 수 있다.
- AlexNet 이후 VGGNet과 GoogLeNet에서도 깊이의 증가가 성능 향상의 중요한 요인으로 나타났다.

2. 기존의 학습 문제: Vanishing Gradient와 Exploding Gradient

- 과거에는 네트워크가 깊어질수록 gradient가 지나치게 작아지거나 커지는 문제가 발생했다.
- Gradient가 지나치게 작아지면 앞쪽 레이어의 weight가 거의 업데이트되지 않는다.
- Gradient가 지나치게 커지면 학습이 불안정해지고 loss가 발산할 수 있다.
- 적절한 weight initialization과 Batch Normalization이 등장하면서 이러한 문제는 상당 부분 완화되었다.
- 따라서 수십 개 이상의 레이어를 가진 네트워크도 학습을 시작하고 수렴시키는 것이 가능해졌다.

3. 새롭게 드러난 문제: Degradation Problem

- Gradient 문제가 완화된 이후에도 네트워크를 일정 수준 이상 깊게 만들면 성능이 오히려 나빠지는 현상이 발생했다.
- Test error가 단순히 포화되는 것이 아니라, 깊은 모델에서 더 높아졌다.
- 더욱 중요한 점은 깊은 모델의 training error도 함께 증가했다는 것이다.

Overfitting의 일반적인 형태:

- Training error는 낮고, Test error만 높다.

Degradation problem의 형태:

- 더 깊은 모델의 training error 자체가 높고, Test error도 함께 높다.
- 따라서 degradation은 데이터에 지나치게 맞춘 overfitting 문제가 아님.
- 네트워크가 깊어지면서 optimizer가 좋은 parameter를 찾지 못하는 optimization 문제이다.

4. 왜 Degradation Problem이 이상한가?

- 얇은 모델에 여러 레이어를 추가한 깊은 모델을 생각해볼 수 있다.
- 추가된 레이어들이 아무런 변환도 하지 않고 identity mapping을 수행한다면, 깊은 모델은 기존의 얇은 모델과 동일한 함수를 표현할 수 있다.
- 즉 깊은 모델은 최소한 얇은 모델과 같은 training error를 달성할 수 있어야 한다.
- 그런데 실제 plain network에서는 깊은 모델의 training error가 더 높았다.

- 이는 optimizer가 추가된 레이어를 identity mapping으로 만드는 쉬운 해조차 제대로 찾지 못한다는 뜻이다.

5. Residual Learning의 제안

일반적인 neural network는 원하는 함수 $H(x)$ 를 직접 학습한다.

ResNet은 학습 대상을 다음과 같이 바꾼다.

$$F(x) = H(x) - x$$

따라서 다음 관계가 성립한다.

$$H(x) = F(x) + x$$

- $H(x)$: 해당 블록이 최종적으로 구현해야 하는 함수
- x : 블록에 들어온 입력
- $F(x)$: 입력 x 에 추가해야 하는 변화량

Plain block의 관점:

- 입력 x 를 완전히 새로운 표현 $H(x)$ 로 변환한다.

Residual block의 관점:

- 입력 x 는 기본적으로 유지한다.
- 입력에 필요한 수정량 $F(x)$ 만 추가한다.
- 최적의 함수 $H(x)$ 가 identity mapping에 가깝다면, 여러 nonlinear layer가 x 를 다시 만들어내는 것보다 $F(x)$ 를 0에 가깝게 만드는 것이 훨씬 쉽다.
- 여기서 쉽다는 것은 연산량이 줄어든다는 의미가 아니다.
- Optimizer가 좋은 parameter를 찾기 쉬운 형태로 문제를 재정의했다는 의미이다.

2. Related Work

1. Residual Representation

- 이미지 표현 분야에서도 원본 값을 직접 사용하는 것보다 기준점으로부터의 차이인 residual을 표현하는 방식이 효과적인 경우가 있었다.
- VLAD는 local feature와 visual dictionary center 사이의 차이 벡터를 인코딩한다.

- Fisher Vector 역시 기준 분포와 feature 사이의 차이를 표현하는 residual 기반 방법으로 볼 수 있다.
- 이러한 사례는 원본 feature 자체보다 기준 대비 변화량을 학습하는 것이 더 효과적일 수 있음을 보여준다.

2. 최적화와 수치해석의 관점

- 수치해석에서는 동일한 문제라도 어떤 형태로 표현하느냐에 따라 수렴 속도가 크게 달라진다.
- Multigrid method와 preconditioning은 문제를 그대로 해결하기보다, 더 풀기 쉬운 형태로 재구성한다.
- ResNet 역시 모델이 표현할 수 있는 함수의 범위를 크게 바꾸는 것이 아니라, 학습할 함수를 residual 형태로 재정의한다.
- 즉 ResNet의 핵심은 network architecture의 변화이면서 동시에 optimization problem의 형태 재구성이다.

3. Shortcut Connection 계열

- 여러 레이어를 건너뛰는 shortcut 또는 skip connection은 ResNet 이전에도 사용되었다.
- GoogLeNet은 중간 레이어에 auxiliary classifier를 연결했다.
- Highway Network는 shortcut 경로를 조절하는 학습 가능한 gate를 사용했다.
- Highway Network의 shortcut은 gate 값에 따라 닫히거나 약해질 수 있다.
- 반면 ResNet의 identity shortcut은 별도의 parameter가 없고 항상 열려 있다.
- 입력은 shortcut을 통해 그대로 전달되고, residual branch는 그 위에 필요한 변화량만 추가한다.

ResNet의 차별점:

- Parameter-free identity shortcut → 연산 없이 바로 shortcut
- Residual만 학습하는 단순한 구조
- 추가적인 gate 학습이 필요하지 않음

3. Deep Residual Learning

3.1 Residual Learning

- 여러 레이어로 이루어진 블록이 최종적으로 학습해야 하는 함수를 $H(x)$ 라고 하자.
- 일반적인 plain network는 $H(x)$ 를 직접 근사한다.
- ResNet은 다음과 같이 residual function을 정의한다.

$$F(x) = H(x) - x$$

- 따라서 원래 학습하려던 함수는 다음과 같이 표현된다.

$$H(x) = F(x) + x$$

- $H(x)$ 가 identity mapping에 가깝다면 plain network는 여러 convolution과 activation을 통과하면서 x 를 다시 만들어야 한다.
- ResNet에서는 residual branch의 출력을 0에 가깝게 만들기만 하면 $H(x)$ 가 x 에 가까워진다.
- 실제 최적 함수가 정확한 identity mapping이 아니더라도, identity에서 조금 변화한 함수라면 전체 함수를 처음부터 학습하는 것보다 수정량만 학습하는 것이 쉽다.

핵심 해석:

- Plain network: 입력을 새로운 표현으로 완전히 변환한다.
- Residual network: 입력을 유지하면서 필요한 정보만 수정한다.

주의할 점:

- Residual은 모델의 예측값과 정답 사이의 prediction error를 의미하지 않는다.
- 여기서 residual은 블록이 원하는 함수 $H(x)$ 와 입력 x 사이의 차이다.

3.2 Identity Mapping by Shortcuts

Residual block의 기본식은 다음과 같다.

$$y = F(x, \{W_i\}) + x$$

- x : residual block의 입력
- $F(x, \{W_i\})$: convolution layer들이 학습하는 residual mapping
- $\{W_i\}$: residual branch의 학습 가능한 weight
- y : residual branch와 shortcut branch를 더한 결과

2-layer residual block을 단순화하면 다음과 같이 표현할 수 있다.

$$F(x) = W_2 \sigma(W_1 x)$$

따라서 전체 블록은 다음과 같다.

$$y = W_2 \sigma(W_1 x) + x$$

- W_1 : 첫 번째 convolution layer의 weight
- W_2 : 두 번째 convolution layer의 weight
- σ : ReLU와 같은 nonlinear activation function

Original ResNet에서는 다음 순서로 연산한다.

Conv → BN → ReLU → Conv → BN → Addition → ReLU

1. 입력과 출력의 차원이 같은 경우

Residual branch의 출력과 shortcut branch의 입력 크기가 같으면 다음과 같이 더할 수 있다.

$$y = F(x) + x$$

x 의 shape: [B, 64, 56, 56] + $F(x)$ 의 shape: [B, 64, 56, 56]

→ 출력 shape: [B, 64, 56, 56]

- 두 tensor를 같은 위치와 같은 채널끼리 element-wise addition
- Identity shortcut에는 추가 parameter가 필요하지 않는다.
- Addition을 제외하면 추가적인 연산량도 거의 없다.

2. 입력과 출력의 차원이 다른 경우

Residual branch와 shortcut branch의 shape이 다르면 바로 더할 수 없다.

x 의 shape: [B, 64, 56, 56] + $F(x)$ 의 shape: [B, 128, 28, 28] = 😬

이런 경우, shortcut branch에 projection을 적용한다.

$$y = F(x) + W_{sx}$$

- W_{sx} : shortcut의 차원을 변환하는 projection
- 일반적으로는 1×1 convolution을 사용
- spatial resolution도 줄여야 한다면 stride = 2를 적용

Projection 이후:

W_{sx} 의 shape: [B, 128, 28, 28]

따라서 다음 addition이 가능하다.

$$F(x) + W_{sx}$$

3. Zero-padding 방식

- 채널 수가 증가하는 경우 기존 입력 채널 뒤에 0을 추가하여 차원을 맞출 수 있다.
- 추가 parameter가 필요하지 않는다.
- Spatial resolution이 감소하는 구간에서는 shortcut에도 stride를 적용한다.

주의할 점:

- Identity mapping과 zero-padding은 완전히 동일한 개념은 아니다.
- 차원이 같은 경우에는 입력을 그대로 전달하는 순수 identity shortcut을 사용한다.
- 차원이 다른 경우에는 projection 또는 zero-padding이 필요하다.

3.3 Network Architectures

1. Plain Network

- VGGNet의 설계 철학을 참고하여 대부분 3×3 convolution을 사용한다.
- 동일한 spatial resolution을 유지하는 동안에는 채널 수를 동일하게 유지한다.
- Feature map의 가로와 세로가 절반으로 감소하면 채널 수를 2배로 늘린다.
- Downsampling에는 stride = 2 convolution을 사용한다.
- 마지막에는 Global Average Pooling을 적용한다.
- 이후 Fully Connected Layer와 softmax를 통해 ImageNet의 1000개 class를 예측한다.

Convolution의 연산량은 대략적으로 다음 값에 비례함.

$$H \times W \times C_{in} \times C_{out} \times K^2$$

- H, W: feature map의 높이와 너비
- C_{in} : 입력 채널 수
- C_{out} : 출력 채널 수
- K: kernel 크기

Feature map의 가로와 세로를 각각 절반으로 줄이면 spatial area는 1/4이 된다.

입력 채널과 출력 채널을 각각 약 2배로 늘리면 채널 관련 계산량은 약 4배가 된다.

따라서 spatial size를 절반으로 줄이고 채널 수를 2배로 늘리면 stage별 convolution 연산량을 비슷하게 유지할 수 있다.

2. Residual Network

- Residual network는 plain network와 거의 동일한 convolution 구조를 사용한다.
- 차이는 여러 convolution layer를 묶은 블록에 shortcut connection을 추가한다는 점이다.
- 차원이 같은 경우에는 identity shortcut을 사용한다.
- 차원이 달라지는 stage 전환에서는 zero-padding 또는 1×1 projection을 사용한다.

3. VGGNet과의 비교

- ResNet-34는 VGG-19보다 훨씬 깊다.
- 하지만 연산량은 VGG-19보다 적다.
- ResNet-34의 연산량은 약 3.6 billion FLOPs이다.
- VGG-19의 연산량은 약 19.6 billion FLOPs이다.
- ResNet은 깊이를 늘리면서도 stage 설계, channel 구성, Global Average Pooling 등을 통해 연산량을 효율적으로 유지한다.

3.4 Implementation

1. Image Preprocessing과 Data Augmentation

- 이미지의 짧은 변을 256에서 480 사이의 크기로 무작위 조절한다.
- 이를 scale augmentation이라고 한다.
- 변환된 이미지에서 224×224 영역을 무작위 crop한다.
- 이미지를 좌우 반전하여 사용할 수 있다.
- Per-pixel mean subtraction을 적용한다.
- Color augmentation도 함께 사용한다.

2. Batch Normalization

- 각 convolution 바로 다음에 Batch Normalization을 적용한다.
- BN 다음에 ReLU를 적용한다.
- Original ResNet의 일반적인 residual branch는 다음과 같다.

Conv → BN → ReLU → Conv → BN → Addition → ReLU

- BN은 activation과 gradient의 분포를 안정적으로 유지하는 데 도움을 준다.
- Original ResNet 실험에서는 dropout을 따로 사용하지 않았다.
- 다만 이것이 ResNet에서는 dropout을 사용할 수 없다는 뜻은 아니다.

3. Optimization 설정

- Optimizer: SGD
- Mini-batch size: 256
- Initial learning rate: 0.1
- Error가 정체되면 learning rate를 1/10로 감소
- Weight decay: 0.0001
- Momentum: 0.9
- 최대 약 600,000 iterations 학습
- ReLU에 적합한 weight initialization 사용

4. Testing 설정

- 기본 비교 실험에서는 standard 10-crop testing을 사용한다.
- 최고 성능 실험에서는 fully convolutional testing을 사용한다.
- 여러 image scale에서 얻은 prediction score를 평균내는 multi-scale testing도 적용한다.

4. Experiments

4.1 ImageNet Classification

ImageNet 2012 데이터셋을 사용하여 plain network와 ResNet의 성능을 비교했다.

ImageNet 2012:

- 약 1000개의 object class
- 약 120만 장의 training image
- Classification 성능 평가에 Top-1 error와 Top-5 error 사용

1. Plain Network와 ResNet 비교

Plain Network 결과

- 34-layer plain network가 18-layer plain network보다 training error가 더 높았다.
- Validation error 역시 34-layer 모델이 더 높았다.
- 깊이가 증가했음에도 학습 데이터 자체를 더 잘 맞히지 못했다.
- 이는 overfitting이 아니라 optimization 실패를 의미한다.

Vanishing Gradient와의 차이

- 두 plain network 모두 Batch Normalization을 사용했다.
- Forward activation의 분산과 backward gradient의 크기가 정상적으로 유지되는 것을 확인했다.
- 따라서 이 실험에서 발생한 degradation을 단순한 vanishing gradient로 설명하기는 어렵다.
- 학습 시간을 늘려도 깊은 plain network의 높은 training error는 해결되지 않았다.

ResNet 결과

- ResNet에서는 34-layer 모델이 18-layer 모델보다 training error가 낮았다.
- Validation error 역시 34-layer 모델이 더 낮았다.
- 동일한 깊이의 plain network와 비교해도 ResNet의 성능이 훨씬 좋았다.
- ResNet-18과 plain-18의 최종 성능 차이는 크지 않았지만, ResNet-18이 학습 초기에 더 빠르게 수렴했다.

결론:

- Residual learning은 깊은 모델의 degradation을 완화한다.

- 비교적 얇은 모델에서도 optimization 속도를 개선할 수 있다.
- 깊이를 증가시켰을 때 추가된 레이어가 실제 성능 향상으로 이어지게 한다.

2. Identity Shortcut과 Projection Shortcut 비교

논문에서는 세 가지 shortcut 방식을 비교했다.

Option A: Zero-padding Shortcut

- 차원이 같으면 identity shortcut 사용
- 채널 수가 증가하면 zero-padding 사용
- 추가 parameter 없음

Option B: Projection When Needed

- 차원이 증가하거나 spatial resolution이 감소하는 stage 전환에서만 1×1 projection 사용
- 나머지 블록에서는 identity shortcut 사용

Option C: All Projection

- 모든 shortcut에 1×1 projection 사용
- 차원이 같은 블록에서도 projection 적용

결과

성능은 대체로 다음 순서로 나타났다.

Option C > Option B > Option A

- 하지만 세 방식의 성능 차이는 크지 않았다.
- 세 방식 모두 plain network보다 훨씬 좋은 성능을 보였다.
- Projection shortcut 자체가 degradation 해결의 핵심은 아니었다.
- Option C는 모든 블록에 추가 parameter와 연산량이 발생한다.
- 이후 깊은 모델 실험에서는 효율성을 고려해 주로 Option B를 사용했다.

3. Bottleneck Architecture

ResNet-50 이상의 모델에서는 연산 효율을 위해 bottleneck block을 사용한다.

구조:

$1 \times 1 \text{ Conv} \rightarrow 3 \times 3 \text{ Conv} \rightarrow 1 \times 1 \text{ Conv}$

각 convolution의 역할:

- 첫 번째 1×1 convolution: 채널 수 축소
- 3×3 convolution: 축소된 채널 공간에서 spatial feature 추출
- 마지막 1×1 convolution: 채널 수 복원 또는 확장

예시:

입력: 256 channels

중간 축소: 64 channels

3×3 연산: 64 channels

출력 복원: 256 channels

Tensor shape 예시:

$[B, 256, 56, 56]$

$\rightarrow [B, 64, 56, 56]$

$\rightarrow [B, 64, 56, 56]$

$\rightarrow [B, 256, 56, 56]$

- 비용이 큰 3×3 convolution을 64채널 공간에서 수행하므로 연산량이 크게 감소한다.
- Bottleneck이라는 이름은 최종 출력 채널 수가 작다는 뜻이 아니다.
- 중간의 3×3 convolution이 좁은 channel space에서 수행된다는 뜻이다.

4. ImageNet 결과

- ResNet-50, ResNet-101, ResNet-152 모두 깊이가 증가할수록 성능이 향상되었다.
- ResNet-152는 VGG-19보다 약 8배 깊다.
- 하지만 연산량은 VGG-19보다 낮다.
- ResNet-152의 연산량은 약 11.3 billion FLOPs이다.
- ResNet-152 단일 모델의 Top-5 validation error는 4.49%였다.
- 서로 다른 깊이의 6개 모델을 결합한 ensemble은 Top-5 test error 3.57%를 기록했다.
- 이 결과로 ILSVRC 2015 classification 부문에서 1위를 차지했다.

4.2 CIFAR-10 and Analysis

CIFAR-10 실험에서는 단순한 성능 경쟁보다, 100층과 1000층 이상의 극단적으로 깊은 네트워크가 어떻게 동작하는지를 분석했다.

CIFAR-10:

- 32×32 크기의 이미지
- 총 10개의 class
- Training image 50,000장
- Test image 10,000장

1. CIFAR-10용 ResNet 구조

Feature map 변화:

$$32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8$$

Channel 변화:

$$16 \rightarrow 32 \rightarrow 64$$

각 stage에는 $2n$ 개의 convolution layer가 배치된다.

전체 weighted layer 수는 다음과 같다.

$$\text{전체 layer 수} = 6n + 2$$

예시:

- $n = 3 \rightarrow \text{ResNet-20}$
- $n = 5 \rightarrow \text{ResNet-32}$
- $n = 7 \rightarrow \text{ResNet-44}$
- $n = 9 \rightarrow \text{ResNet-56}$
- $n = 18 \rightarrow \text{ResNet-110}$
- $n = 200 \rightarrow \text{ResNet-1202}$

모든 shortcut에는 parameter-free Option A를 사용했다.

이렇게 한 이유:

- Plain network와 ResNet의 깊이, 너비, parameter 수를 최대한 동일하게 유지한다.
- 성능 차이가 shortcut과 residual learning에서 발생했다는 점을 명확하게 확인하기 위해서이다.

2. 깊이에 따른 성능 변화

Plain Network

- 깊이가 증가할수록 training error가 증가했다.
- ImageNet에서 관찰된 degradation problem이 CIFAR-10에서도 반복되었다.
- Plain-110은 training error가 60%를 넘을 정도로 학습에 실패했다.

ResNet

- ResNet-20에서 ResNet-56까지 깊이가 증가할수록 training error와 test error가 감소했다.
- ResNet-110은 약 1.7M parameter로 6.43% test error를 기록했다.
- 매우 깊은 network도 residual connection을 통해 안정적으로 optimization할 수 있음을 확인했다.

3. Layer Response Analysis

논문의 가설:

- Residual branch가 학습하는 $F(x)$ 는 상대적으로 작은 변화량일 것이다.
- 네트워크가 깊어질수록 각 block은 입력을 크게 바꾸기보다 작은 수정만 수행할 것이다.

분석 방법:

- 각 3×3 convolution의 BN 이후, activation 이전 출력값의 표준편차를 측정했다.
- Plain network와 ResNet의 layer response 크기를 비교했다.

결과:

- ResNet의 residual response는 동일한 깊이의 plain network보다 전반적으로 작았다.
- ResNet-20, ResNet-56, ResNet-110을 비교하면 깊은 모델일수록 개별 residual branch의 response가 작아지는 경향을 보였다.
- 즉 모델이 깊어질수록 각 블록은 입력에 작은 수정만 추가했다.

해석:

- Residual learning이 의도한 방향과 일치한다.
- 다만 이것이 모든 residual output이 정확히 0에 가깝다는 수학적 증명은 아니다.

- 실험적으로 residual response가 plain network의 response보다 작게 나타났다는 의미이다.

4. ResNet-1202

- $n = 200$ 으로 설정하여 1202-layer ResNet을 학습했다.
- Training error는 0.1% 미만까지 감소했다.
- 즉 1000층이 넘는 모델도 optimization 자체에는 큰 문제가 없었다.
- 하지만 test error는 7.93%로 ResNet-110보다 높았다.
- ResNet-1202의 parameter 수는 약 19.4M이었다.
- CIFAR-10처럼 작은 데이터셋에 비해 모델이 지나치게 커 overfitting이 발생한 것으로 해석했다.

결론:

- Residual connection은 매우 깊은 모델의 optimization을 가능하게 한다.
- 하지만 깊이가 증가한다고 항상 test 성능이 향상되는 것은 아니다.
- 데이터 크기, model capacity, regularization을 함께 고려해야 한다.

4.3 Object Detection on PASCAL VOC and MS COCO

Classification에서 학습한 ResNet feature가 다른 CV task에서도 효과적인지 검증했다.

1. 실험 설정

- Detection framework: Faster R-CNN
- 기존 backbone: VGG-16
- 비교 backbone: ResNet-101
- 나머지 detection pipeline은 최대한 동일하게 유지했다.

따라서 성능 차이는 주로 backbone이 추출한 feature representation의 차이로 해석할 수 있다.

2. PASCAL VOC 결과

VOC 2007 mAP:

- VGG-16: 73.2
- ResNet-101: 76.4

VOC 2012 mAP:

- VGG-16: 70.4
- ResNet-101: 73.8

3. COCO 결과

COCO mAP@0.5:

- VGG-16: 41.5
- ResNet-101: 48.4

COCO mAP@[0.5:0.95]:

- VGG-16: 21.2
- ResNet-101: 27.2

해석:

- COCO의 표준 mAP 지표가 21.2에서 27.2로 6.0 percentage points 상승했다.
- 상대적 성능 향상은 약 28%이다.
- 낮은 IoU threshold뿐 아니라 엄격한 IoU threshold에서도 성능이 향상되었다.
- 이는 ResNet feature가 object class를 구분하는 능력뿐 아니라 bounding box를 정확하게 localization하는 능력도 개선했음을 의미한다.